408 逐题拆解 · 2022 年数据结构选择题（第 1–11 题）

配合《408 诊断-2022 选择题》使用：数据结构这次对 3、错 2、空白 6。拆法和英语逐词翻译一样——把每道题涉及的知识点拆成小块，先讲人话再给术语，最后逐个选项排雷。你不翻王道书了，这份拆解就当教材用，定义、公式、结论都在小块里交代完整。

第 1 题 · 双重循环的时间复杂度

【题目】 程序段：外层 `for (i=1; i<=n; i*=2)`，内层 `for (j=0; j<i; j++) sum++`，求时间复杂度。正确答案 **B (O(n))**。你这次：⬜ 空白。

【考点小块】

小块 1 · 复杂度数的是"执行次数"，不是"循环层数"。概念：时间复杂度看基本操作（这里是 sum++）总共执行多少次，和 n 是什么数量级关系。套路：内层执行次数由外层变量 i 当时的值决定，i 取 1、2、4、8……（每轮翻倍），所以总次数 = 1 + 2 + 4 + … 一直加到不超过 n 的最大 2 的幂。

小块 2 · 等比数列求和的直觉。1 + 2 + 4 + … + 2^k = 2^(k+1) − 1，其中 2^k ⩽ n，所以总和 < 2n+1。也就是说总次数被 2n 摁住，是 n 的常数倍 → O(n)。易错：看到外层只跑 $\log_2 n$ 次就选 O(log n)，忘了内层每次跑的量在涨；或者把内层当成"每次都跑 n 次"，误选 O(n log n)。记住翻倍型循环的总量由**最后一项的量级**决定：最后一项约 n，前面所有项加起来还不到它的 1 倍。

【逐项排雷】

A：$O(\log_2 n)$ 只数了外层圈数，没数内层干的活

B：✓ 总次数 1+2+4+…≈2n，O(n)

C：$O(n \log_2 n)$ 是把内层误当成每轮都跑满 n 次

D：$O(n^2)$ 毫无依据，内层最多跑 n 次、外层才 log 次

【一句话结论】 翻倍循环别逐层乘：把每轮的执行次数排成数列求和，总量约等于最后一项的 2 倍。

【同类题指针】 循环次数求和是历年复杂度题的主力考法（翻倍、递减、双指针都要会排数列求和），回测时留意同类题。

第 2 题 · 栈的入栈/出栈序列

【题目】 in 和 out 是同一批元素的两个排列，对初始为空的栈 ST，判断四条叙述哪条正确。正确答案 **D**。你这次：✅ 对（选 D）。

【考点小块】

小块 1 · 出栈序列怎么判合法。概念：给定入栈顺序 in，任一时刻只能弹出"最后入栈且还没出栈"的那个元素。套路：模拟——按 in 依次入栈，每入一个就看栈顶是不是 out 里下一个该出的，是就弹出；最后能把 out 完整对上就是合法序列。所以**给定 in，能判断 out 是否可能**，A 说"不能判断"是错的。同理，给定 out 反推 in 也可验证，B 同样错。

小块 2 · 两个极端情形。入栈后立刻出栈（边入边出），out 与 in 完全相同——所以 C 说"in 与 out 一定不同"错。全入完再依次出，out 恰好是 in 的倒序——所以 D 说"可能互为倒序"对。易错：把"可能"看成"必然"：倒序只是众多合法出栈序列中的一种。

【逐项排雷】

A：能判断（模拟即可），错

B：同样能判断，错

C：边入边出时 in 与 out 可以相同，错

D：✓全部入栈再依次出栈，out 就是 in 的倒序

【一句话结论】 栈的序列题只认两件事：合法性靠模拟验证，倒序和全同都是合法序列的极端情形。

【同类题指针】 栈序列合法性判断（给 in 判 out、卡特兰数计数）在历年真题中反复出现，回测时留意。

第 3 题 · 中序遍历中相邻结点的关系

【题目】 p、q 在二叉树中序遍历序列中相邻且 p 在 q 之前，问 p 与 q 的关系中**不可能**的是：I. q 是 p 的双亲；II. q 是 p 的右孩子；III. q 是 p 的右兄弟；IV. q 是 p 的双亲的双亲。正确答案 **B（仅 III）**。你这次：❌错，选了 D（仅 II、IV），把可能的两项判成了不可能，又漏了真正不可能的右兄弟。

【考点小块】

小块 1 · 中序遍历的顺序骨架。中序 = 左子树 → 根 → 右子树。推相邻关系只靠一条线索：一个结点的中序"前任/后任"是谁。规律：结点 x 的中序后任 = ①若 x 有右子树，是右子树中最左的结点；②若 x 没有右子树，往上找，直到某个祖先是"从左孩子上去的"，那个祖先就是后任。

小块 2 · 逐项造例验证（这类题只能靠造小树验证，不能凭感觉）。

I 可能：p 是 q 的左孩子且 p 无右孩子——中序先走 p，紧接着就是根 q。

II 可能：q 是 p 的右孩子且 q 无左孩子——中序走完 p，右子树第一个访问的就是 q 本人。

III 不可能：q 是 p 的右兄弟，中间必定隔着它们的双亲——中序顺序是"p 所在的左子树 → 双亲 → 右子树"，双亲永远夹在两兄弟之间，两人不可能紧挨着。

IV 可能：p 是祖父 q 左子树里中序最后访问的结点（例如 q 的左孩子 m 以 p 为右孩子、p 无右子树，中序依次是 m、p、q）——左子树走完，紧接着就是根（祖父）q。

易错点（正是你的错）：直觉上觉得"孩子、祖父离得远，不会相邻"，但中序的相邻看的是**遍历顺序**不是辈分距离；反而看着最近的右兄弟，因为中间隔着双亲，永远不相邻。

【逐项排雷】

A：仅 I——I 明明可能，错

B：✓仅 III 不可能

C：含 II——II 可以造出例子，错

D：含 II、IV——两项都可能，你选的就是这个

【一句话结论】 中序相邻只看遍历顺序：双亲、右孩子、祖父都能紧挨，唯独右兄弟中间隔着双亲，永远不相邻。

【同类题指针】 遍历序列推结点关系（前任/后任、由两种遍历定树）在历年树题中高频出现，回测时留意。

第 4 题 · 三叉树的最小高度

【题目】 三叉树有 244 个结点（叶结点高度为 1），求树的高度至少是几。正确答案 **C（6）**。你这次：✅ 对（先倾向 D 的 5，最后改成 C，改对了）。

【考点小块】

小块 1 · "高度至少"的问法 = 先算容量。概念：高度为 h 的树能装多少结点有上限；要装下 244 个，高度必须够大。问"至少"，就是找最小的 h 使"h 高的树最多能装的结点数 ⩾ 244"。套路口径先对齐：叶结点高度为 1，即高度数的是层数。

小块 2 · m 叉树满装公式。高度 h 的三叉树装得最满时每层都满：第 1 层 1 个、第 2 层 3 个、第 3 层 9 个……总数 = 1 + 3 + … + 3^(h−1) = (3^h − 1)/2。代入试：h=5 → (243−1)/2 = 121 < 244，装不下；h=6 → (729−1)/2 = 364 ⩾ 244，装得下。所以至少 6。易错：套成二叉树的 2^h − 1；或者把"至少"理解反，去算最少结点数（那对应的是链状树，没有意义）。

【逐项排雷】

A：8——容量早就够了，"至少"不取大

B：7——同上，不是最小

C：✓ h=5 最多 121 个不够，h=6 才够

D：5——满装才 121 个，244 个塞不进去（你最初倾向的就是它）

【一句话结论】 "n 个结点的 m 叉树高度至少"一律先算满装容量 (m^h − 1)/(m − 1)，找第一个装得下的 h。

【同类题指针】 树的结点数与高度互推（二叉树版 (2^h − 1)、完全二叉树版）是历年常客，回测时留意。

第 5 题 · 哈夫曼编码树 vs 定长编码树

【题目】 同一字符集 S（n > 2 个字符），用二叉树表示哈夫曼编码集得 T_1、定长编码集得 T_2，问哪条叙述正确。正确答案 **D**。你这次：⬜ 空白。

【考点小块】

小块 1 · 两棵树的长相完全不同。定长编码：每个字符编码长度相同（$\lceil \log_2 n \rceil$ 位），对应一棵"整齐"的二叉树——所有字符（叶子）都在同一层，树高由 n 决定，和字符出现频次无关。哈夫曼编码：按频次建树，频次高的离根近、频次低的离根远，树可以歪，叶子分布在不同层。

小块 2 · 哈夫曼树的结点数是定值。哈夫曼建树每次把两个最小的合并成一个新结点，n 个叶子恰好合并 n−1 次，产生 n−1 个内部结点，总数恒为 2n − 1。而定长编码树的结点数取决于补齐成满树后的规模，一般不等于 2n − 1，所以 A 说"结点数相同"没有依据。

小块 3 · "频次不同 → 层不同"是个陷阱。哈夫曼只保证总加权路径最短，不保证频次不同就分层：频次 2 和频次 3 的字符完全可能挂在同一层（深度相同、编码一样长）。反过来，定长编码里不管频次差多少，编码长度一律相同，叶子必然同层——这正是 D。

【逐项排雷】

A：T_1 恒为 $2n-1$ 个结点，T_2 不一定，错

B：T_1 的高度可大可小（频次均匀时和 T_2 差不多，频次悬殊时更高），"大于"说死了，错

C：频次不同的字符在 T_1 中可以同层，错

D：✓ 定长编码人人等长，叶子必在同一层

【一句话结论】 哈夫曼看频次（结点数恒 2n−1、叶子不同层），定长看个数（叶子必同层、与频次无关）。

【同类题指针】 哈夫曼树结点数、WPL 计算、编码长度是历年高频考点，回测时留意。

第 6 题 · 无向图连通与边数的关系

【题目】 对无向图 G=(V, E)，判断关于 |V|（顶点数）与 |E|（边数）的四条说法哪条正确。正确答案 **D**。你这次：⬜ 空白。

【考点小块】

小块 1 · 连通的最低边数门槛。一个连通无向图至少要有 |V| − 1 条边（恰好 |V| − 1 条边的连通图就是树，一条边都不能再少）。推论：**|E| < |V| − 1 时，图一定不连通**——边不够把所有顶点串起来。这是本题唯一能"一定"的方向。

小块 2 · 边多 ≠ 一定连通。边数超过门槛，只能说"可能连通"，判不了"一定"：完全可以把很多边堆在一个分量里形成环，再扔一个孤立顶点在旁边，边再多照样不连通。看到选项里"边多所以一定连通"，立刻用"有环分量 + 孤立点"这个反例去砸。

小块 3 · 套到各选项。D：|V| > |E| + 1 等价于 |E| < |V| − 1，正中门槛，一定不连通 ✓。C：|V| = |E| − 1 即 |E| = |V| + 1，边比树多两条——既能连通（在树上加两条边），也能不连通（有环分量 + 孤立点凑数），"一定不连通"错。A：|V| > |E| 只说明边比顶点少，连通（树，|E| = |V| − 1 满足此式）与不连通都有可能。B：|V| < |E| 边很多，同样两种可能都有。

【逐项排雷】

A：树就满足 |V| > |E| 且连通，"一定连通"也推不出（边更少时又不连通），错

B：有环分量 + 孤立点边再多也不连通，错

C：|E| = |V| + 1 时连通与不连通都能构造，错

D：✓ |E| < |V| − 1，边数不足，一定不连通

【一句话结论】 连通只认一条硬线：边数 < 顶点数 − 1 必不连通；边再多也只能说可能连通。

【同类题指针】 图的基本量关系（连通分量、生成树边数、度数总和）是选择题常客，回测时留意。

第 7 题 · AOE 网中时间余量最大的活动

【题目】 图中 AOE 网有 10 个活动 a–j，求时间余量最大的活动。正确答案 **B（活动 g）**。你这次：✅ 对（选 B）。

【考点小块】

小块 1 · 先分清两个"时间"。AOE 网里顶点是事件、边是活动，边权是活动耗时。每个事件有两个量：最早发生时间 ve（从源点正推，取 max）和最迟发生时间 vl（从汇点倒推，取 min）。每个活动的余量 = 它最迟必须开始的时刻 − 最早可以开始的时刻 = vl(终点) − ve(起点) − 边权。余量 0 的活动就是关键活动，拖了它整个工期就拖。

小块 2 · 本题实算。顶点 1–6。正推 ve：ve(1)=0，ve(2)=2，ve(3)=max(5, 2+3)=5，ve(4)=max(2+3, 5+3)=8，ve(5)=max(5+3, 8+1)=9，ve(6)=max(8+4, 5+1, 9+1)=12（工期 12）。倒推 vl：vl(6)=12，vl(5)=11，vl(4)=min(12−4, 11−1)=8，vl(3)=min(8−3, 11−3, 12−1)=5，vl(2)=min(5−3, 8−3)=2，vl(1)=0。逐活动算余量：a=0、b=0、c=0、d=3、e=0、f=3、g=11−5=**6**、h=2、i=0、j=2。g 余量 6，最大。

易错：把 ve/vl 搞混方向（正推取 max、倒推取 min，反了全盘错）；或者拿顶点的时间差当余量，忘了再减边权。关键路径不止一条：本题 a、b、c、e、i 余量都是 0。

【逐项排雷】

A：c 是关键活动，余量 0，最小而非最大

B：✓ g 的余量 6，全场最大

C：h 的余量 2，不及 g

D：j 的余量 2，不及 g

【一句话结论】 AOE 余量题只有一条路：正推 ve 取 max、倒推 vl 取 min、余量 = vl(终) − ve(始) − 权，关键活动余量为 0。

【同类题指针】 关键路径/AOE 与 AOV 拓扑排序在历年真题中轮换出现（含大题），回测时留意。

第 8 题 · 5 阶 B 树删除关键字后的根

【题目】 5 阶 B 树根为 [60, 90, 260, 350]，五个孩子依次为 [30, 50]、[70, 80, 85]、[100, 110]、[280, 300]、[400, 500]。删除关键字 260 后经必要调整得新树 T_1，问哪个**不可能是** T_1 的根关键字序列。正确答案 **D（60, 90, 110, 350）**。你这次：❌ 错，选了 B（60, 90, 350）——B 其实可能，你漏了一种调整走向。

【考点小块】

小块 1 · B 树的硬指标（5 阶）。每个结点最多 4 个关键字；除根外，每个结点至少 ⌈5/2⌉ − 1 = 2 个关键字。删除后任何结点少于 2 个就叫"亏空"，必须调整。

小块 2 · 删内部结点的套路。260 在根（内部结点）里，不能直接抠掉：用它的**前驱**（左边子树里的最大值 = 左孩子 [100, 110] 里的 110）或**后继**（右边子树里的最小值 = 右孩子 [280, 300] 里的 280）顶上来，再去叶子层删那个被借走的值。叶子被删后若亏空，只有两招：

借：某个兄弟有 ⩾3 个关键字时，父结点关键字下沉补亏，兄弟的最大（或最小）关键字上顶到父结点——根会被换掉一个关键字。

合并：兄弟都只有 2 个、借不出时，把亏空结点 + 一个兄弟 + 中间的父关键字合并成一个大结点——根会少一个关键字。

小块 3 · 本题三种真实走向（260 左右孩子都只有 2 个关键字，是关键条件）。

走前驱 110：根先变 [60, 90, 110, 350]；叶子 [100, 110] 删 110 后只剩 [100]，亏空。左兄弟 [70, 80, 85] 有 3 个、借得出：父关键字 90 下沉补成 [90, 100]，兄弟最大值 85 上顶——根变成 [60, **85**, 110, 350]，即选项 C，可能。

走后继 280：根先变 [60, 90, 280, 350]；叶子 [280, 300] 删 280 后只剩 [300]，亏空。左右兄弟都只有 2 个，借不出，只能合并：往左与 [100, 110] 合并、父关键字 280 下沉，根丢掉 280 → 根变 [60, 90, 350]，即选项 B，可能（你误判的就是这条）；往右与 [400, 500] 合并、父关键字 350 下沉，根丢掉 350 → 根变 [60, 90, 280]，即选项 A，可能。

选项 D [60, 90, 110, 350]：根里出现 110 只能是走了前驱；但前驱一走，叶子必亏空，后续无论借（85 上顶）还是合并（90 下沉），根都不可能保持"60、90 原样 + 多一个 110、350 不动"——所以 D 不可能。

【逐项排雷】

A：60, 90, 280——后继顶替 + 向右合并的结果，可能

B：60, 90, 350——后继顶替 + 向左合并的结果，可能（你误杀了它）

C：60, 85, 110, 350——前驱顶替 + 向左兄弟借的结果，可能

D：✓ 60, 90, 110, 350——110 上来了根却没付出任何代价，不可能

【一句话结论】 B 树删内部关键字：前驱/后继顶上，叶子亏空必借或必并——根要么被换一个关键字，要么少一个关键字，没有白嫖的调整。

【同类题指针】 B 树插入/删除后的形态判断是历年树章节的难点常客，回测时留意。

第 9 题 · 影响散列平均查找长度的因素

【题目】 问哪些因素影响散列（哈希）方法的平均查找长度（ASL）：I. 装填因子；II. 散列函数；III. 冲突解决策略。正确答案 **D（I、II、III 全影响）**。你这次：⬜ 空白。

【考点小块】

小块 1 · ASL 是什么。平均查找长度 = 平均查找一次要比较（探测）多少次。它只由一件事决定：**冲突多不多、冲突了之后找起来费不费劲**。凡是能改变冲突概率或冲突后处理成本的因素，都影响 ASL——本题三个选项恰好各占一个环节。

小块 2 · 逐个看三个因素。

I 装填因子 α = 表中元素个数 ÷ 表长。α 越大，表越满，新元素越容易撞车，探测次数越多。α 是 ASL 公式里的直接变量（如线性探测成功 ASL ≈ ½(1 + 1/(1−α))），α→1 时 ASL 暴涨。

II 散列函数。函数把关键字分布得越均匀，扎堆越少，冲突越少；设计得差（比如只取奇偶）会人为制造大量冲突。

III 冲突解决策略。开放定址（线性探测容易"聚集"）与链地址法（同义词挂链表）处理冲突的方式不同，同样数据下 ASL 不同——线性探测的聚集效应就是典型例子。

易错：以为 ASL 只由装填因子决定（把公式里的 α 当成唯一因素），漏掉 II、III。公式里只有 α，是因为公式默认函数均匀、策略已定；题目问"哪些因素影响"，要把三个环节全算上。

【逐项排雷】

A：漏了 III，错

B：漏了 II，错

C：漏了 I，错

D：✓ 三个环节全影响

【一句话结论】 散列 ASL 三决定：装填因子管"满不满"，散列函数管"匀不匀"，冲突策略管"撞了怎么找"。

【同类题指针】 ASL 计算（成功/不成功、不同冲突策略）是历年查找章节的计算常客，回测时留意。

第 10 题 · 二路归并操作的功能

【题目】 用二路归并排序对数组 M 排序时，问"二路归并操作"的功能是什么。正确答案 **A（将两个有序表合并为一个新的有序表）**。你这次：⬜ 空白。

【考点小块】

小块 1 · 先把"归并排序"和"归并操作"分开。归并排序全过程是三步：①划分——把表对半拆，递归拆到每段只剩 1 个元素（天然有序）；②递归处理左右两半；③归并——把两个已经有序的表合成一个有序表。题目问的"二路归并操作"单指第 ③ 步这一个动作，"二路"就是"两路输入"的意思。

小块 2 · 归并动作怎么做。两个指针分别指着两个有序表的头，谁小先取谁放进结果表，一方取完后把另一方剩下的整段抄过去。一趟归并比较次数至多是两表长度之和减一，时间 O(n)。易错：B 说的"划分成大致相等的两部分"是第 ① 步划分，不是归并；C 说的"划成 n 个单元素部分"是递归拆分的终点状态，也不是归并；D 说的"一部分值全小于另一部分"是快速排序里枢轴划分的效果，张冠李戴——归并的两半之间没有任何大小关系，左半的最大值可以大于右半的最小值。

【逐项排雷】

A：✓ 归并 = 两个有序表合成一个有序表

B：那是划分步骤，不是归并操作

C：那是递归拆分的终点，不是归并操作

D：那是快速排序的枢轴划分效果，串科了

【一句话结论】 归并操作只做一件事：把两个有序表合并成一个有序表；划分是它的前一步，枢轴分区是快排的活。

【同类题指针】 各排序算法的过程辨析（划分、堆调整、一趟结果）在历年排序题中高频出现，回测时留意。

第 11 题 · 选直接插入排序而不选快速排序的理由

【题目】 对数据排序时，若采用直接插入排序而不采用快速排序，可能的原因是：I. 大部分元素已有序；II. 待排序元素数量很少；III. 要求空间复杂度为 O(1)；IV. 要求排序算法是稳定的。正确答案 **D（I、II、III、IV 全部成立）**。你这次：⬜ 空白。这题是"一条都不能漏"的全选题，逐条讲清。

【考点小块】

小块 1 · 先把两个算法的底细摆齐。直接插入排序：最好 O(n)（基本有序时）、最坏 $O(n^2)$、平均 $O(n^2)$；辅助空间 O(1)；**稳定**。快速排序：平均 O(n log n)，但最坏 $O(n^2)$（基本有序时若选首元素作枢轴，每次划分都极不均衡，退化最严重）；递归需要栈空间，平均 O(log n)、最坏 O(n)；**不稳定**。

小块 2 · 逐条判定。

I 成立：大部分元素已有序时，直插每插一个几乎不用挪动，接近 O(n)；而快排在近有序数据上最容易退化到 $O(n^2)$。数据越有序，越该选直插。

II 成立：元素很少时，$O(n^2)$ 和 O(n log n) 的实际差距小到可忽略，直插实现简单、常数开销小、没有递归调用成本，反而更快。

III 成立：直插只需要 1 个临时单元，空间严格 O(1)；快排靠递归实现，栈空间平均 O(log n)、最坏 O(n)，满足不了"必须是 O(1)"的硬要求。

IV 成立：稳定性 = 相等元素的相对次序不变。直插遇到相等元素时把新元素放在已有元素后面，次序不变，稳定；快排的远距离交换会把相等元素顺序打乱，不稳定。有稳定性要求时只能排除快排。

易错：III 最容易被漏——很多人只记得快排"平均空间 O(log n)"就觉得它省空间，忘了题目要求是严格的 O(1)，且快排最坏要 O(n)。IV 也常被犹豫：记住八大排序里稳定的四个——插入、冒泡、归并、基数。

【逐项排雷】

A：漏了 III、IV，错

B：漏了 I、II，错

C：漏了 III——快排的递归栈使它达不到 O(1)，错

D：✓四条全部成立

【一句话结论】 直插赢快排靠四点：近有序近 O(n)、小数据开销小、空间严 O(1)、稳定；快排的软肋正是有序退化、要栈空间、不稳定。

【同类题指针】 排序算法多维对比（时间/空间/稳定性/适用场景）几乎每年必考，回测时留意。

回测清单（隔 2 天，10-23 重做）

重做题号：**第 1、2、3、4、5、6、7、8、9、10、11 题（全部 11 题）**。

使用说明：

1. 做对的题（2、4、7）：先自己重做一遍再看，确认每个干扰项都知道错在哪——不是蒙对、不是只记住答案字母，才算真过关。第 4 题特别注意：你这次是临场改对的，重做时要能直接写出 (3^h − 1)/2 的容量算法。

2. 错题（3、8）：先合上这份拆解，自己把原题重做一遍，做完再打开拆解核对答案和错因——先做后看，才测得出是不是真会了。第 3 题重做时逐项造小树验证，第 8 题把"前驱/后继 × 借/合并"的走向在草稿上全推一遍。

3. 空白题（1、5、6、9、10、11）：先只看每题的【考点小块】把知识补上，然后合上材料回做原题，能独立选对才算这一题过关。

408 逐题拆解 · 2022 年计组选择题（第 12–22 题）

配合《408 诊断-2022 选择题》使用：计组这次 0 对、4 错、7 空白，丢的全是基础题。拆法和英语逐词翻译一样——把每道题涉及的知识点拆成小块，先讲人话再给术语，最后逐个选项排雷。第 13、14、19 题的知识点已在《408 计组 D1 数据表示亲手算训练》里成套练过，本稿简写，标注了"配合 D1"。

第 12 题 · CPI 与 CPU 执行时间

【题目】 主频 1GHz，共执行 10000 条指令，80% 的指令平均 1 个周期、20% 平均 10 个周期，求平均 CPI 和 CPU 执行时间。正确答案 **A（2.8，28μs）**。你这次：❌ 错，选了 D（28，28ms）。

【考点小块】

小块 1 · 平均 CPI。概念：CPI 是"平均每条指令花几个时钟周期"，几类指令混在一起就按占比加权，像算加权平均分。套路：$0.8\times1+0.2\times10=2.8$。易错：结果必然落在 1 和 10 之间，算出 28 先怀疑小数点错位。

小块 2 · 执行时间与单位。总周期数 = 指令条数 × CPI = 10000 × 2.8 = 28000；主频 1GHz → 一个周期 1ns；时间 = 28000ns = 28μs。易错：单位阶梯 $1s=10^3\ ms=10^6\ \mu s=10^9\ ns$，每步差 1000，ms 和 μs 最常翻车（你的 28000 其实算对了，死在最后一步）。

【逐项排雷】

A：CPI 2.8、时间 28μs，两项都对 ✓

B：CPI 错成 28（小数点错位），时间对

C：CPI 对，时间错成 28ms（差了 1000 倍）

D：两项全错——你选的这个，就是 CPI 和单位同时翻车

【一句话结论】 CPI 先加权平均，时间 = 条数 × CPI ÷ 主频，选答案前默念一遍单位。

【出处】 王道计组 第 1 章 计算机系统概述

第 13 题 · 32 位补码整数范围（简写 · 配合 D1 模块一）

【题目】 问 32 位补码所能表示的整数范围。正确答案 **B（$-2^{31}\sim2^{31}-1$）**。你这次：⬜ 空白。

【考点小块】

补码范围。概念：补码的编码像一个圆环，正数占一半少一个、负数占一半多一个，因为 -2^{n-1} 没有对应的正数镜像。套路：n 位 → $[-2^{n-1},2^{n-1}-1]$，n = 32 直接套。易错：原码/反码是对称范围 $\pm(2^{n-1}-1)$，别和补码混。

【逐项排雷】

A：下限 -2^{32} 错——32 位总共只有 2^{32} 个编码，不可能表示到 -2^{32}

B：✓ 与公式完全吻合

C：把无符号范围 $0\sim2^{32}-1$ 硬套了正负号

D：上限 $2^{32}-1$ 错，正数最大只能到 $2^{31}-1$

【一句话结论】 n 位补码范围 $[-2^{n-1}, 2^{n-1}-1]$，负数那边多一个。

【出处】 王道计组 第 2 章 数据的表示和运算（完整推导与练习见 D1 训练模块一）

第 14 题 · IEEE 754 单精度表示（简写 · 配合 D1 模块二）

【题目】 把 −0.4375 写成 IEEE 754 单精度浮点数的十六进制表示。正确答案 **A（BEE0 0000H）**。你这次：⬜ 空白。

【考点小块】

浮点数拼装。概念：单精度 = 1 位符号 + 8 位阶码（真指数 + 127）+ 23 位尾数（藏起整数位的那个 1）。套路：$0.4375 = 0.0111_2 = 1.11 \times 2^{-2}$ → 符号 1；阶码 = −2 + 127 = 125 = 01111101；尾数取小数点后的"11"再补 0；拼出 1011 1110 1110 0000⋯ = BEE0 0000H。易错：阶码忘 +127、尾数把整数位 1 也写进去，是两个最高频坑。

【逐项排雷】

A：✓ 三段全部拼对

B：BF60 0000H 对应 −0.875——规格化时小数点少移一位（写成了 1.11×2^{-1}）

C：BF70 0000H 对应 −0.9375——阶码和尾数都错位拼出来的

D：C0E0 0000H 对应 −7——指数的方向弄反了

【一句话结论】 浮点拼装三步：化二进制规格化 → 阶码 +127 → 尾数去掉领头的 1 补齐 23 位。

【出处】 王道计组 第 2 章 数据的表示和运算（正反两道母题见 D1 训练模块二）

第 15 题 · 虚存地址变换

【题目】 分页虚存，页大小 4KB，访问虚拟地址 00082840H，页表给出：虚页号 82→页框 024H（存在位 0）、129→180H（存在位 1）、130→018H（存在位 1），问转换结果。正确答案 **C（得到主存地址 01 8840H）**。你这次：⬜ 空白。

【考点小块】

小块 1 · 切地址。概念：虚拟地址像"楼号 + 房号"，高位是虚页号、低位是页内偏移，偏移占几位由页大小说了算。套路：页大小 4KB = 2^{12} → 低 12 位是偏移 = 840H；高位是虚页号 = 00082H。易错：82H 是十六进制，等于十进制 **130**，查表要查 130 那一行，这是全题最大的坑。

小块 2 · 查表拼地址。概念：页表就是"虚页号 → 页框号"的对照表，存在位 0 表示这页还没进内存。套路：130 行页框号 018H、存在位 1 → 物理地址 = 页框号后移 12 位再接偏移 = 018840H，写成 01 8840H。易错：018H 和 180H 长得像，慢看一位。

【逐项排雷】

A：02 4840H 用的是 82 行的页框 024H——查错行，何况那行存在位是 0

B：18 0840H 用的是 129 行的页框 180H——看串了一行（这就是本题的经典陷阱）

C：✓ 页号 130、页框 018H、偏移 840H，拼接正确

D：缺页异常是把虚页号当成十进制 82、查到存在位为 0 那一行才会得出的结论

【一句话结论】 先按页大小切出页号和偏移，页号是十六进制先换算，查对行后页框号原样拼回偏移。

【出处】 王道计组 第 3 章 存储系统

第 16 题 · Cache 比较器个数与位数

【题目】 Cache 数据区 32KB、主存块 64B、8 路组相联，求比较器的个数和位数。正确答案 **A（8 个，20 位）**。你这次：⬜ 空白。

【考点小块】

小块 1 · 行数与组数。概念：先数 Cache 一共有多少"行"（每行装一个主存块），再按每组几行分组。套路：行数 = 32KB ÷ 64B = 512 行；每组 8 行 → 组数 = 512 ÷ 8 = 64 组。易错：给的是数据区容量，直接除块大小就是行数，不要把标记位算进去。

小块 2 · 地址三段切分。概念：主存地址 = 标记 + 组号 + 块内偏移。套路：块内偏移 = $\log_2 64 = 6$ 位；组号 = $\log_2 64 = 6$ 位；标记 = 32 − 6 − 6 = **20 位**。易错：组号位数由组数决定，不是由"路数"直接变的。

小块 3 · 比较器。概念：访存时拿地址里的标记，和被选中那一组里每一行的标记同时比对，几行就几个比较器。套路：比较器个数 = 路数 = 8，每个比 20 位。易错：64 是组数——64 组是"选 1 组"，不是"同时比 64 个"。

【逐项排雷】

A：8 个、20 位，都对 ✓

B：个数对；位数 23 是把组号误算成 3 位（拿"8 路"直接开 $\log_2$）凑出来的

C：位数对；个数 64 是把组数冒充成了比较器

D：个数、位数都错

【一句话结论】 比较器个数 = 路数，位数 = 标记位数 = 总位数 − 组号 − 块内偏移。

【出处】 王道计组 第 3 章 存储系统

第 17 题 · DRAM 内存条

【题目】 内存条由 8 个 8192×8192×8 位 DRAM 芯片组成，总线 64 位、支持突发传送，每芯片有一个行缓冲，问哪条叙述**不正确**。正确答案 **C**。你这次：⬜ 空白。

【考点小块】

小块 1 · 容量怎么数。概念：8 片是"分工拼宽度"——每片出 8 位，8 片并排正好凑 64 位，地址空间只有一份。套路：一片有 8192×8192 = 2^{26} 个单元，并行后每个地址读出 8B → 总容量 = 2^{26} × 8B = **512MB**。易错：8 片相加的是"宽度"不是"深度"，别把容量乘多一份。

小块 2 · 行缓冲与地址引脚。概念：DRAM 内部是方阵，访问先把一整行搬进缓冲；行、列地址分两次、**共用同一组引脚**送入，省引脚。套路：8192 = 2^{13}，行列各 13 位但复用 → 地址引脚 13 根；行缓冲 = 一整行 = 8192×8 位。易错：看到"行列各 13 位"就加成 26，恰恰忘了"复用"二字。

【逐项排雷】

A：512MB，算法如上 ✓

B：多模块交叉编址 ✓（官方口径判对：多片并行组织，支撑 64 位总线宽度与突发传送）

C：说引脚 26 位 ✗——行列复用只有 13 根，这就是要找的"不正确"

D：行缓冲 8192×8 位，就是一整行的大小 ✓

【一句话结论】 DRAM 记牢三件事：容量按"单元数 × 位宽"、行缓冲等于一整行、地址引脚行列复用只算一份。

【出处】 王道计组 第 3 章 存储系统

第 18 题 · ISA 规定的内容

【题目】 问哪些属于指令集体系结构（ISA）规定：I 指令字格式和指令类型、II CPU 的时钟周期、III 通用寄存器个数和位数、IV 加法器的进位方式。正确答案 **B（仅 I、III）**。你这次：⬜ 空白。

【考点小块】

ISA 的分界线。概念：ISA 是"软件能看见的合同"——写程序、写编译器需要知道的归它管；电路内部怎么实现、软件看不见的，不归它管。套路：逐项问一句"写程序的人需要知道吗？"需要 → 是 ISA；不需要 → 是实现（微体系结构）。易错：寄存器的个数和位数程序员必须知道（指令里要给寄存器编号、运算按位宽来），归 ISA；时钟周期、进位电路纯属造 CPU 的事。

【逐项排雷】

A：含 II——时钟周期是实现参数，同一种 ISA 可以有不同主频的芯片，错

B：I、III 都是软件可见面 ✓

C：选的全是实现细节，还漏了 I、III

D：多含 IV——加法器用串行还是并行进位，指令层面完全无感

【一句话结论】 判断 ISA 只看一条线：软件看得见的是 ISA，看不见的是实现。

【出处】 王道计组 第 4 章 指令系统

第 19 题 · 扩展操作码（简写 · 配合 D1 模块三）

【题目】 16 位定长指令、地址码 6 位，二地址指令 12 条、一地址指令 254 条，求零地址指令最多几条。正确答案 **D（128）**。你这次：❌ 错，选了 B（2）——会算，但停早了一步。

【考点小块】

扩展操作码两步走。概念：像电话号段分配——上一级用剩的每个前缀，都能展开成一整段下一级号码。套路：二地址操作码 4 位共 16 个前缀，用 12 剩 4 → 一地址容量 = $4\times2^6=256$ 条；用 254 剩 2 个前缀 → 零地址 = $2\times2^6=$ **128** 条。易错（正是你的错）：算到"剩 2 个前缀"就停手，忘了每个前缀还要再展开 2^6。

【逐项排雷】

A：0 是以为号段被一地址占满了

B：2 就是你停手的位置——那是剩余**前缀数**，不是指令条数

C：64 只相当于展开了 1 个前缀

D：✓ 两步走完：$2 \times 2^6 = 128$

【一句话结论】 扩展操作码永远两步：剩余前缀数 × 2 的地址码位数次方，少乘一步必错。

【出处】 王道计组 第 4 章 指令系统（母题 + 3 道亲手算见 D1 训练模块三，建议重做本题收尾）

第 20 题 · 源程序到可执行文件的过程

【题目】 问将高级语言源程序转换为可执行目标文件的主要过程。正确答案 **A（预处理 → 编译 → 汇编 → 链接）**。你这次：❌ 错，选了 B，把编译和汇编记反了。

【考点小块】

四道工序。概念：像流水线加工——预处理是备料（展开头文件、宏替换）；编译是把"人话菜谱"翻成汇编语言（半成品）；汇编是把汇编语言翻成机器码目标文件（零件）；链接是把多个零件和库拼成整机（可执行文件）。套路：按产物倒推顺序——编译的产物是汇编程序，汇编的产物才是机器码，所以编译必在汇编之前。易错：名字带"汇"的离机器码最近，离可执行文件只差一步链接，不可能排在编译前面。

【逐项排雷】

A：✓ 顺序全对

B：编译与汇编对调——编译的产物就是汇编代码，对调后逻辑讲不通（你选的）

C：把链接提到汇编之前——还没有目标文件，无可链接

D：编译、汇编、链接三步全乱

【一句话结论】 预处理备料、编译出汇编、汇编出机器码、链接拼整机，顺序不能换。

【出处】 王道计组 第 1 章 计算机系统概述

第 21 题 · 中断 I/O 方式

【题目】 问关于中断 I/O 方式哪条叙述**不正确**。正确答案 **C**。你这次：❌ 错，选了 B——你以为错的那条恰恰是对的。

【考点小块】

小块 1 · 中断 I/O 怎么工作。概念：像点外卖——外设是慢厨房，备好一个字就按一次门铃（发中断），CPU 放下手头活、执行中断服务程序把这个字取走，再回去干活。所以传送动作是 **CPU 用软件完成的**。套路：抓三个特征：慢速字符设备适用、CPU 与外设并行、每次只搬少量数据。易错：和 DMA 混——DMA 才是硬件成批搬运、CPU 基本不插手，中断方式不是。

小块 2 · 时间大小关系。概念：门铃响一次的处理必须远快于备下一份餐的时间，否则 CPU 要被门铃淹没。套路：外设准备数据时间（慢，毫秒级）**远大于**中断处理时间（快，微秒级）。易错：题面把关系写成"应小于"，就是埋好的错述。

【逐项排雷】

A：✓ 键盘、针式打印机正是典型的慢速字符设备

B：✓ 数据传送通过软件（CPU 执行中断服务程序）完成，这是中断方式与 DMA 的分界线——你误判在这一条

C：✗ 把大小关系说反了，准备时间应远**大于**处理时间——这就是要找的"不正确"

D：✓ 外设备餐时 CPU 去运行其他进程，正是中断方式的意义所在

【一句话结论】 中断 I/O = 慢设备、软件搬运、并行工作；准备时间必须远大于处理时间。

【出处】 王道计组 第 7 章 输入输出系统

第 22 题 · 并行处理技术

【题目】 问关于并行处理技术哪条叙述**不正确**。正确答案 **C**。你这次：⬜ 空白。

【考点小块】

小块 1 · Flynn 分类法。概念：按"指令流几条 × 数据流几条"给机器分型。套路：多核 = 多个核各跑各的指令、各处理各的数据 → MIMD；向量处理器 = 一条指令同时算一串数据 → SIMD。易错：向量机的"多"是数据多，不是核多，别划进 MIMD。

小块 2 · 硬件多线程与 SMP。概念：硬件多线程靠多套寄存器等硬件上下文实现快速切换，和有几个核无关，单核照样能做；SMP（对称多处理）是多个处理器对等、共享同一个物理地址空间。套路：概念题里出现"只、仅、必须"这类绝对说法，先打一个问号。易错：把"多核常见"误当成"单核不行"。

【逐项排雷】

A：✓ 多核处理器是 MIMD 的典型

B：✓ 向量处理器是 SIMD 的典型

C：✗ 单核处理器靠多套线程上下文就能实现硬件多线程，"只可用于多核"说错了——要找的就是它

D：✓ 共享单一物理地址空间正是 SMP 的定义特征

【一句话结论】 Flynn 看指令流 × 数据流；硬件多线程不挑核数，单核也能玩。

【出处】 王道计组 第 1 章 计算机系统概述

回测清单（隔 2 天，10-23 重做）

重做题号：**第 12、13、14、15、16、17、18、19、20、21、22 题（全部 11 题）**。

使用说明：

1. 错题（12、19、20、21）：先合上这份拆解，自己把原题重做一遍，做完再打开拆解核对答案和错因——先做后看，才测得出是不是真会了。

2. 空白题（13、14、15、16、17、18、22）：先只看每题的【考点小块】把知识补上，然后合上材料回做原题，能独立选对才算这一题过关。

408 逐题拆解 · 2022 年操作系统选择题（第 23–32 题）

配合《408 诊断-2022 选择题》使用：OS 这次对 1、错 4、空白 5。错的 4 道里有 2 道（23、27）是"概念知道、答案没改彻底"，真正不会的是缺页和银行家这类流程题。这份拆解按"以后不再翻王道书"的标准写——每道题涉及的知识点都讲全，先讲人话再给术语，最后逐个选项排雷。

第 23 题 · 多道程序系统与"抖动"

【题目】 问关于多道程序系统哪条叙述**不正确**。正确答案 **D（进程数越多 CPU 利用率越高）**。你这次：❌ 错，括号里写了 B——但其实你在 D 旁边打了勾，第二判断是对的，只是没把答案改过来。

【考点小块】

小块 1 · 多道程序设计是什么。概念：内存里同时放几道程序，A 等磁盘的时候 CPU 去跑 B，靠"错峰"把 CPU 喂饱，这就是多道程序设计。套路：它的标志是**并发**（宏观上同时推进）和**资源共享**（CPU、内存、设备大家轮流用，必须有管理者来分配）——对应 A、C 都是真叙述。易错：多道 ≠ 多核，多道是单核上也能玩的错峰术。

小块 2 · 多道程序不需要虚存。多道只要求"几道程序能同时装进内存"，早期系统靠分区、覆盖、交换就能做到，虚拟存储是后来才加的豪华配置。所以 B"不必支持虚拟存储管理"是真叙述。易错：别把"现代系统都有虚存"倒推成"多道必须有虚存"。

小块 3 · 抖动（颠簸）。进程不是越多越好：进程太多时，每个进程分到的页框太少，刚调进来的页马上又被换出去，CPU 大部分时间在忙着换页而不是干活，这叫**抖动**，此时进程数再增加，CPU 利用率反而**下降**。D 把这句话说成单调上升，就是错述。

【逐项排雷】

A：✓ 并发执行正是多道的定义

B：✓ 多道不以虚存为前提，分区/交换也能实现多道（你写了它，但它是真叙述，不能选）

C：✓ 共享必然需要管理，否则资源会打架

D：✗ 进程过多会抖动，利用率不升反降——要找的就是它（你其实在它旁边打了勾：**相信第二判断，改答案就改彻底，括号和涂卡一起改**）

【一句话结论】 多道靠错峰提利用率，但进程过多会抖动，利用率过犹不及。

【同类题指针】 "抖动/颠簸"常和虚拟内存、工作集的题绑在一起出现，回测时留意同一考点的判断题。

第 24 题 · 操作系统初始化时创建什么

【题目】 问下列哪样东西需要在操作系统**初始化（启动）过程中**创建：中断向量表 / 文件系统根目录 / 硬盘分区表 / 文件系统索引节点表。正确答案 **A（中断向量表）**。你这次：⬜ 空白。

【考点小块】

小块 1 · 分清"开机现建"和"早就存好"。概念：电脑里的管理结构分两类——一类只存在于内存里、断电就没，必须每次开机现建；另一类是写在磁盘上的、断电不丢，是在分区、格式化（装系统）时就建好的，开机只是把它们读进来用。套路：判断"是否在 OS 初始化时创建"，就问一句：**它是住在内存里还是住在磁盘上？**住内存的 → 开机建；住磁盘的 → 装机时建。

小块 2 · 中断向量表是什么。它是一张"中断号 → 处理程序入口地址"的电话簿，放在内存固定位置。CPU 一遇到中断（比如键盘敲击、时钟到点），就按中断号去查这张表、跳过去处理。这张表断电消失，所以每次开机由 OS 初始化代码亲手建好、填好。没有它，中断根本没法响应——所以它必须在开机阶段就位。

【逐项排雷】

A：✓ 内存中的表，OS 启动时创建并填写

B：✗ 根目录是文件系统的一部分，格式化建文件系统时就写在磁盘上了，不用每次开机重建

C：✗ 分区表是给磁盘分区时写进磁盘最前面扇区的，开机时 OS 只是读取它，不是创建它

D：✗ 索引节点（inode）表同样是创建文件系统时在磁盘上划好的区域，开机不重建

【一句话结论】 开机现建的只有内存里的管理表；分区表、根目录、inode 表都是装机时写在磁盘上的。

【同类题指针】 这类"OS 引导/初始化"概念题频率不高，回测时留意即可，判断钥匙就是"内存 or 磁盘"。

第 25 题 · 优先级抢占调度：调度次数

【题目】 四个进程 P0（0ms 到达，优先级 15，需 100ms）、P1（10ms，20，60ms）、P2（10ms，10，20ms）、P3（15ms，6，10ms），优先级**数值越小越优先**，抢占式调度，问从 0ms 到全部结束共发生进程调度多少次。正确答案 **C（6 次）**。你这次：⬜ 空白（草稿画了但没画完）。

【考点小块】

小块 1 · 抢占式优先级调度怎么跑。规则只有两条：① 任何时刻，CPU 给就绪队列里优先级最高的进程；② 新进程到达时若优先级高于正在跑的，立刻把正在跑的踹下去（抢占），被踹的进程回就绪队列等下次。套路：这类题唯一可靠的办法是**画时间线**，盯住三个事件点——新进程到达、进程跑完、被抢占，别凭感觉数。

小块 2 · "调度次数"怎么数。每"选一个进程上 CPU"算 1 次调度，**包括开局的第一次上台，也包括被抢占的进程后来重新上台**。易错：很多人只数"切换次数"（相邻两段之间换人），会比调度次数少 1；还有人把同一进程的两段合并，直接漏数。

小块 3 · 逐步推演（时间线）。

0ms：只有 P0 在场 → P0 上台，开始跑它的 100ms。

10ms：P1（20）、P2（10）到达，P2 的 10 比 P0 的 15 高 → 抢占。P0 已跑 10ms、还剩 90ms，回队列；P2 上台。

15ms：P3（6）到达，全场最高 → 抢占 P2。P2 只跑了 5ms、还剩 15ms；P3 上台。

25ms：P3 的 10ms 跑完结束。队列里 P2（10）最高 → P2 第二次上台，接着跑剩下的 15ms。

40ms：P2 跑完结束。队列剩 P0（15）、P1（20），P0 高 → P0 第二次上台，跑剩下的 90ms。

130ms：P0 跑完结束，只剩 P1 → P1 上台，跑 60ms，190ms 全部结束。

合计上台 6 次 → 调度 6 次。

【逐项排雷】

A：4 次——只数了"4 个进程各上台一次"，漏了 P2、P0 被抢占后的二次上台

B：5 次——数了切换次数（6 段之间 5 个接缝），或漏了开局那次

C：✓ 时间线共 6 段，每段一次调度

D：7 次——把进程到达也误算成了一次调度，到达本身不算，只有"上台"才算

【一句话结论】 抢占调度画时间线，调度次数 = 进程上台的总段数，含开局、含回锅。

【同类题指针】 调度计算是 OS 选择题的常驻户（优先级、RR、SJF 轮换着考），回测时留意每年 OS 的计算型选择题，画时间线的套路通用。

第 26 题 · 银行家算法：数安全序列

【题目】 三个进程、三类资源 A/B/C。已分配：P0(2,0,1)、P1(0,2,0)、P2(1,0,1)；尚需 Need：P0(0,2,1)、P1(1,2,3)、P2(0,1,3)；当前可用 Available(1,3,2)。问此时安全序列有几个。正确答案 **B（2 个）**。你这次：⬜ 空白。

【考点小块】

小块 1 · 安全序列是什么。概念：银行家算法是"防死锁"的预判法。一种排队顺序叫安全序列，如果按这个顺序，每个进程轮到时，它还差的资源（Need）都能被"当前可用 + 前面进程释放的"凑够，它就能跑完并把自己占的资源全吐出来。能找出至少一个安全序列，系统就是安全的。套路：维护一个工作向量 Work，初始 = Available；反复找 Need ≤ Work（逐个分量都不超过）的进程，假设它跑完，Work += 它的 Allocation；能把所有进程排完，就得到一个安全序列。

小块 2 · 本题第一步就被卡死。Work = (1,3,2)，逐个验：P0 的 Need(0,2,1) 全 ≤ ✓；P1 的 Need(1,2,3) 里 C 要 3，可用只有 2 ✗；P2 的 Need(0,1,3) 同样卡在 C ✗。**所以第一个跑完的只能是 P0**，这一步没有分叉。P0 释放 (2,0,1) 后 Work = (3,3,3)。

小块 3 · 第二步开始分叉，枚举到底。Work = (3,3,3) 时 P1(1,2,3) ✓、P2(0,1,3) ✓ 都能跑，出现两个分支：分支一 P1 先：P1 释放 (0,2,0)，Work = (3,5,3)，P2 的 (0,1,3) ✓ 能跑完 → 序列 ⟨P0, P1, P2⟩；分支二 P2 先：P2 释放 (1,0,1)，Work = (4,3,4)，P1 的 (1,2,3) ✓ 能跑完 → 序列 ⟨P0, P2, P1⟩。两条路都通到底，安全序列共 **2** 个。易错：找到一个安全序列就停手——题目问的是"几个"，必须把每个分叉都枚举完；另一个常见错是 Need 和 Allocation 用反（释放时加的是已分配 Allocation，不是 Need）。

【逐项排雷】

A：1 个——只找到一条序列就收工，漏了第二步的分叉

B：✓ ⟨P0,P1,P2⟩ 与 ⟨P0,P2,P1⟩，共 2 个

C：3 个——多半是把 P1 或 P2 也算成了可打头（它俩第一步就被 C 资源卡死）

D：4 个——把排不通的顺序也算了进去

【一句话结论】 数安全序列：Work 从可用资源起步，Need 不超 Work 才能跑、跑完加回它的已分配，每个分叉都要枚举到底。

【同类题指针】 银行家算法是 OS 选择题常客，常以"判断状态是否安全 / 数安全序列 / 某请求能否批准"三种问法出现，回测时留意。

第 27 题 · CPU 模式：用户态与内核态

【题目】 问关于 CPU 模式哪条叙述正确。正确答案 **C（CPU 处于用户态时只能执行非特权指令）**。你这次：❌ 错，括号里写了 B——但你在选项旁已经把答案改标成了 C，又是概念对了、卷面没改彻底。

【考点小块】

小块 1 · 两种模式分什么。概念：CPU 有两副面孔——**内核态**（管态）是"管理员模式"，任何指令都能执行；**用户态**（目态）是"访客模式"，只能执行普通计算指令。操作系统内核跑在内核态，你写的应用程序跑在用户态。套路：记一句话——**用户态是"只能"，内核态是"都能"**。"只能执行非特权指令"配用户态，"能执行一切指令"配内核态，凡是给内核态安"只能"的都是错述。

小块 2 · 特权指令长什么样。特权指令 = 一旦用错会危害整个系统的指令：开关中断、置时钟、修改页表基址、I/O 指令等。非特权指令 = 加减乘除、读写自己内存、跳转这类人畜无害的。用户态若私自执行特权指令，硬件会当场拦下并触发异常。易错：别以为"内核态只能执行特权指令"——内核也要跑普通计算，它是什么都能执行，只是"特权指令只有它能执行"。

【逐项排雷】

A：✗ 把用户态的本事说反了——用户态恰恰碰不了特权指令

B：✗ 内核态能执行**全部**指令（特权 + 非特权），"只能执行特权指令"错在"只能"（你括号里选的就是它）

C：✓ 用户态只能执行非特权指令

D：✗ 内核态能执行非特权指令，而且天天都在执行，D 把关系说反了

【一句话结论】 用户态"只能"非特权，内核态"都能"执行；见到给内核态配"只能"直接排除。

【同类题指针】 这组概念常和系统调用、中断处理"谁在什么态下运行"绑在一起考，回测时留意换皮题。

第 28 题 · 什么事件让进程从执行态变阻塞态

【题目】 下列哪些可能让进程 P 由执行态变为阻塞态：I 进程 P 读文件、II 时间片用完、III 进程 P 申请外设、IV 进程 P 执行信号量的 wait() 操作。正确答案 **D（仅 I、III、IV）**。你这次：❌ 错，选了 C——你把 I 划掉了，以为读文件不会阻塞。

【考点小块】

小块 1 · 三状态转换的判别钥匙。进程状态就三个主角：执行、就绪、阻塞。判别钥匙是问一句：**它是"在等某样东西"，还是"只差一个 CPU"？** 执行 → 阻塞：进程主动等一件还没发生的事（数据没读来、设备没空出、资源被别人占着），等的时候占着 CPU 也没用，所以睡过去，这叫阻塞。执行 → 就绪：进程什么都不缺、只缺 CPU，是被"赶下台"的（时间片用完、被更高优先级抢占），它随时能接着跑，这叫就绪。易错高发区就在"读文件"：读文件 = 向磁盘发 I/O 请求，磁盘比 CPU 慢几个数量级，数据没回来之前进程只能睡等，妥妥的阻塞。

小块 2 · 逐个对号。I 读文件 → 发起 I/O、等数据 → 阻塞 ✓；III 申请外设 → 设备正被别人用（或需要等待分配）→ 阻塞 ✓；IV wait()（即 P 操作）→ 若信号量值不够（资源已被占完），进程主动睡到信号量队列里 → 阻塞 ✓；II 时间片用完 → 被时钟中断赶下台、回就绪队列 → 是变就绪不是阻塞 ✗。

【逐项排雷】

A：仅 I、IV——漏了 III，申请不到外设一样要睡等

B：仅 II、III——II 是就绪不是阻塞，正好选反了主角

C：仅 III、IV——你选的，漏了 I：读文件发起磁盘 I/O，等待期间就是阻塞

D：✓ I、III、IV 都是"主动等事件"，只有 II 是"被赶下台"

【一句话结论】 变阻塞 = 主动等事件（I/O、设备、资源）；变就绪 = 被抢走 CPU（时间片、抢占）。

【同类题指针】 进程状态转换是 OS 最高频的判断点之一，几乎每年都有，回测时留意一切"执行态→？"的箭头题。

第 29 题 · 缺页处理：哪一步"不一定"发生

【题目】 进程访问的页 b 不在内存，触发缺页异常，问缺页处理过程中**不一定包含**哪个操作：淘汰内存中的页 / 建立页号与页框号的对应关系 / 将页 b 从外存读入内存 / 修改页表中页 b 的存在位。正确答案 **A（淘汰内存中的页）**。你这次：⬜ 空白。

【考点小块】

小块 1 · 缺页处理的固定动作。页不在内存 → CPU 暂停、转入 OS 的缺页处理程序。它必做的事是：① 在外存（磁盘）上找到页 b，**把它读进某个页框**（C 必做，页就是这么进来的）；② 在页表里把页 b 和这个页框**登记对应**（B 必做，不然下次还是找不到）；③ 把页 b 的**存在位改成 1**（D 必做，这是"已在内存"的标记）。这三件缺一不可。

小块 2 · 淘汰为什么是"不一定"。读页进来需要一个空页框：如果内存里**正好有空闲页框**，直接用，根本不用淘汰任何页；只有页框全满时，才需要按置换算法挑一页"牺牲品"淘汰（若它是脏页还要先写回外存）。所以淘汰是**有条件才发生**的，正是题目要的"不一定包含"。易错：很多人把"缺页 = 换页"划等号，其实缺页只保证"调入"，调入是否伴随淘汰取决于有没有空框。类比：书不在阅览架上，管理员一定会去库房把书取来、登记好位置；但"先扔掉一本别的书腾位子"只在架子满了时才需要。

【逐项排雷】

A：✓（要找的）有空闲页框时不发生淘汰，只有内存满时才淘汰

B：✗ 必做——不登记映射，这次调入等于白干

C：✗ 必做——页 b 就是靠这一步从外存进内存的

D：✗ 必做——存在位不置 1，下次访问还会再报一次缺页

【一句话结论】 缺页必做三件：调入、登记映射、置存在位；淘汰只在没空框时才发生。

【同类题指针】 缺页流程常和页面置换算法的计算题（缺页次数、置换顺序）成对出现，回测时留意同一章节的组题。

第 30 题 · 什么不影响缺页率

【题目】 问下列哪项**不会影响**系统的缺页率：页面置换算法 / 工作集的大小 / 进程的数量 / 页缓冲队列的长度。正确答案 **D（页缓冲队列的长度）**。你这次：⬜ 空白。

【考点小块】

小块 1 · 缺页率由什么决定。缺页率 = 缺页次数 ÷ 访问总次数，它取决于"访问的页在不在内存里"。能改变这一点的因素：置换算法好不好（算法差，好页被误赶走，回头又缺）；分给进程的页框够不够装下它的**工作集**（进程最近频繁访问的那批页，装不下就反复缺页）；系统里**进程的数量**（进程越多，每人分到的页框越少，缺页率整体上升）。A、B、C 都是从这几个角度改变"页在不在"，全都会影响。

小块 2 · 页缓冲只省时间、不减次数。页缓冲（缓冲队列）是给被淘汰的页留的"后悔药暂存区"：页被淘汰时先不真丢、挂进缓冲队列，若进程马上又访问它，可以直接从队列里捡回来、免去一次磁盘 I/O。但注意——这次访问在统计上**仍然算一次缺页**，它改变的是"这次缺页处理有多贵"（处理代价/时间），不是"缺页发生了几次"。题目问的是**率**，D 只影响代价不影响次数。易错：把"缺页率"和"缺页处理开销"混成一件事，看到"页"字就以为都影响。

【逐项排雷】

A：✗（会影响）置换算法决定赶走谁，直接改变后续缺不缺页

B：✗（会影响）工作集装不进分到的页框，就会频繁缺页

C：✗（会影响）进程越多、人均页框越少，缺页率上升

D：✓ 页缓冲只降低单次缺页的找回代价，不改变缺页发生的次数——要找的就是它

【一句话结论】 缺页率看"页在不在"：算法、工作集、人均页框都影响；页缓冲只省找回时间，不减缺页次数。

【同类题指针】 与第 29 题同属虚拟内存考点群，常和"抖动、工作集、缺页率"打包出现，回测时留意组题。

第 31 题 · 系统调用中哪些由操作系统完成（你做对了，简版）

【题目】 执行系统调用涉及：I 保存断点和程序状态字、II 保存通用寄存器的内容、III 执行系统调用服务程序、IV 将 CPU 模式改为内核态，问其中由**操作系统**完成的是哪些。正确答案 **B（仅 II、III）**。你这次：✅ 对——这道题全国正确率只有约 50%，你拿下了。

【考点小块】

硬件与 OS 的分工线。用户程序执行陷阱（系统调用）指令的那一瞬间，**硬件自动**完成两件事：把断点（PC）和程序状态字（PSW）压栈保存（I），并把 CPU 状态位翻成内核态（IV）——这两步不需要软件插手。等内核接管之后，**OS 软件**才登场：由中断/系统调用处理程序把通用寄存器等现场保存到内核栈或进程控制块（II），再按系统调用号去执行对应的服务程序（III）。套路：凡是"陷入瞬间自动发生的"（存断点、切模式）归硬件；凡是"内核代码一步步执行的"（存寄存器、跑服务程序）归 OS。易错：I 和 IV 最迷惑人，它们确实发生在系统调用过程中，但执行者是硬件不是 OS，题目问的是"谁完成"。

【逐项排雷】

A：仅 I、III——I 是硬件做的，错位

B：✓ II、III 都是内核软件的工作

C：仅 II、IV——IV 也是硬件做的

D：仅 II、III、IV——多带了硬件做的 IV

【一句话结论】 陷入瞬间：存断点、切内核态是硬件自动的；存寄存器、跑服务程序才是 OS 干的。

【同类题指针】 "硬件做还是软件做"是中断、异常、系统调用题的通用判别线，回测时留意同类分工题。

第 32 题 · 驱动程序与 I/O 控制方式

【题目】 问关于驱动程序哪条叙述**不正确**。正确答案 **A（驱动程序与 I/O 控制方式无关）**。你这次：❌ 错，选了 D——其实 D 是对的叙述，错述是 A。

【考点小块】

小块 1 · 驱动程序是干什么的。它是操作系统里最贴近硬件的一层软件：上层只说"读这块盘"，驱动负责把命令翻译成对具体设备控制器的寄存器读写、发起操作、处理完成中断。每种设备一个驱动。套路：判断驱动的题目，抓一条——**驱动是"实际动手操作设备"的那一层**，初始化设备、发读写命令、等设备干完，都是它。

小块 2 · 为什么它和 I/O 控制方式密切相关。I/O 控制方式有四种：程序查询、中断、DMA、通道。同是"读数据"，查询方式下驱动要反复轮询状态位；中断方式下驱动要设置好并等待中断；DMA 方式下驱动要先把缓冲区地址、字节数写进 DMA 控制器、由 DMA 成批搬运。**控制方式变了，驱动要写的代码就完全不同**——所以"驱动与控制方式无关"是彻头彻尾的错话。类比：司机怎么开车，取决于开的是手动挡、自动挡还是电动车，说"驾驶方式与车型无关"显然不对。易错：别把驱动和"设备独立性软件"混了——追求"与设备无关"的是上层接口，不是驱动；驱动恰恰是最不能无关的那一层。

【逐项排雷】

A：✗ 驱动的写法完全取决于控制方式（查询/中断/DMA/通道各写一套）——要找的错述就是它

B：✓ 设备的初始化（清状态、设寄存器、备缓冲）正是由驱动控制完成的

C：✓ 进程执行到驱动里、要等设备完成操作时，可能睡过去进入阻塞态

D：✓ 读/写设备的具体操作就是由驱动操纵控制器完成的——这是对的叙述，你误选了它

【一句话结论】 驱动是最贴硬件的一层：初始化、发命令、等完成都归它，而且控制方式变、驱动就变。

【同类题指针】 I/O 管理里驱动、中断、DMA、通道的辨析题常成组出现，回测时留意这一章节的判断题。

回测清单（隔 2 天，10-23 重做）

重做题号：**第 23、24、25、26、27、28、29、30、31、32 题（全部 10 题）**。

使用说明：

1. 做对的题（31）：直接看逐项确认一遍，重点记住"存断点、切模式归硬件"这条分工线，能复述出来就算过关。

2. 错题（23、27、28、32）：先合上这份拆解，自己把原题重做一遍，做完再打开拆解核对答案和错因——先做后看，才测得出是不是真会了。23、27 额外提醒自己：**改答案要改彻底**，第二判断对了就把括号里的也划掉重写。

3. 空白题（24、25、26、29、30）：先只看每题的【考点小块】把知识补上，然后合上材料回做原题，能独立选对才算这一题过关；25、26 两道计算题务必亲手把时间线和 Work 向量从头推一遍，不许只看结论。

408 逐题拆解 · 2022 年计算机网络选择题（第 33–40 题）

配合《408 诊断-2022 选择题》使用：计网这次对 1、错 2、空白 5。错的 2 道（35、36）全栽在子网计算上，空白的 5 道里有 4 道是"计时/速率"类计算题——不是不会，是没见过这类题的固定套路。拆法和计组那份一样：知识点拆成小块，先讲人话再给术语，最后逐个选项排雷。这份拆解就当教材用，不用再回翻书。

第 33 题 · OSI 模型中哪一层管相邻结点的流量控制

【题目】 在 ISO/OSI 参考模型中，实现两个**相邻结点之间**流量控制功能的是哪一层？正确答案 **B（数据链路层）**。你这次：✅ 对，这是计网唯一做对的一题，本题略讲，确认一下你不是蒙的就行。

【考点小块】

小块 1 · "流量控制"在两层都有，别混。概念：流量控制就是"发方别太快、收方别撑死"，给发送速度装个刹车。OSI 里有两处刹车：**数据链路层**管的是相邻两个结点之间这一段链路（点到点）；**传输层**管的是从源主机到目的主机的全程（端到端）。套路：题目问哪层，先看范围词——"相邻结点"→ 数据链路层；"端到端/两个进程"→ 传输层。易错：看到"流量控制"就反射选传输层，不看范围，这正是本题的埋点。

小块 2 · 顺带把邻居层职能记牢。物理层只管传比特（电压、接口），不管快慢协调；网络层管选路和分组转发，它涉及的是"拥塞控制"这个全网层面的话题，OSI 口径下相邻结点的流量控制不归它。本题只需定位到链路层即可。

【逐项排雷】

A：物理层只传原始比特，没有"控制流量"的概念

B：✓ 相邻结点间 = 点到点，正是数据链路层的地盘（靠滑动窗口等机制实现）

C：网络层管选路，它对应的是拥塞控制，不是相邻结点流量控制

D：传输层确实也做流量控制，但那是端到端的，与题干"相邻结点"范围不符

【一句话结论】 流量控制看范围：相邻结点选链路层，端到端选传输层。

【同类题指针】 OSI/TCP-IP 各层功能归属是计网概念题的常客，回测时留意"某功能属于哪层"这类题，统一用"范围词定位法"处理。

第 34 题 · 奈奎斯特公式求最大数据传输速率

【题目】 带宽 200 kHz 的**无噪声**信道，采用 4 个幅值的 ASK 调制，求信道的最大数据传输速率。正确答案 **C（800 kb/s）**。你这次：⬜ 空白——公式没提取出来，其实只有一行算式。

【考点小块】

小块 1 · 两个公式先分清用哪个。概念：信道能跑多快有两道天花板：**奈奎斯特公式**管"无噪声"的情况，只和带宽、每个码元能带几比特有关；**香农公式**管"有噪声"的情况，要用到信噪比。套路：题干出现"无噪声"→ 直接奈奎斯特；出现"信噪比/dB"→ 香农。本题只给了带宽和调制方式，一个噪声参数都没有，想用香农也没数据。易错：两个公式张冠李戴，或看到"最大速率"就反射香农。

小块 2 · 奈奎斯特公式本体。公式：最大数据传输速率 = 2W · $\log_2 L$（单位 b/s）。W 是带宽（Hz），L 是信号的**进制数/电平数**（能取几种不同状态），$\log_2 L$ 就是"一个码元能装几比特"。人话版：每秒最多变 2W 次信号（这是 Nyquist 给的码元速率上限），每次变化装 $\log_2 L$ 比特，乘起来就是比特率。套路代入：W = 200 kHz，L = 4（4 个幅值 = 4 种状态），$\log_2 4 = 2$，即每个码元装 2 比特；速率 = 2 × 200k × 2 = **800 kb/s**。易错：把 L = 4 直接当 4 比特乘进去（那是 D 的来历）；或者只算 2W = 400 kBaud 就停手——那是码元速率，不是比特率（那是 B 的来历）。

【逐项排雷】

A：200 kb/s——把带宽数值直接当速率，等于默认每秒变 1 次、每次 1 比特，两步都漏

B：400 kb/s——只算了码元速率 2W = 400 kBaud，忘了每个码元还装着 $\log_2 4 = 2$ 比特

C：✓ 2 × 200k × $\log_2 4$ = 800 kb/s，公式和代入都对

D：1600 kb/s——把"4 个幅值"当成"每码元 4 比特"，$\log_2 4$ 应该是 2，不是 4

【一句话结论】 无噪声用奈奎斯特：速率 = 2W · $\log_2 L$，先算"每码元几比特"，别把电平数当比特数。

【同类题指针】 奈奎斯特/香农二选一 + 码元速率与比特率换算，是物理层计算题的固定考点，回测时留意带"信噪比"的版本（那时改用香农公式 C = W · $\log_2(1+S/N)$）。

第 35 题 · 由 IP 和掩码求网络地址（你错的那道）

【题目】 主机 IP 是 183.80.72.48，子网掩码是 255.255.192.0，求该主机所在网络的网络地址。正确答案 **B（183.80.64.0）**。你这次：❌ 错，选了 C（183.80.72.0）——把 IP 的第三字节 72 原样抄进了答案，没做运算。这题的方法诊断报告里已提过，这里把套路立死。

【考点小块】

小块 1 · 网络地址 = IP 与掩码按位与（AND）。概念：IP 地址 = 网络号 + 主机号，掩码就是一把"遮罩"：掩码为 1 的位保留（是网络号），为 0 的位清零（是主机号）。网络地址就是"主机号全清零"后的地址。套路：逐字节看掩码——255 的字节原样抄，0 的字节写 0，只有不 255 不 0 的字节（本题第三字节 192）才需要真算：72 ∧ 192。算一遍：72 = 0100 1000，192 = 1100 0000，逐位与 → 0100 0000 = 64。所以网络地址 = 183.80.**64**.0。易错（正是你的错）：看到第三字节"长得像网络的一部分"就直接抄 72——掩码不是 255 的字节，永远要过一遍与运算。

小块 2 · 块大小法（考场速算，不用写二进制）。概念：掩码某个字节是 M，这个字节就被切成大小为 256 − M 的"块"，网络地址该字节必是块大小的整数倍。套路：第三字节掩码 192 → 块大小 = 256 − 192 = 64 → 边界是 0、64、128、192；72 落在 64 开头的那一块 → 取块首 64。答案秒出。易错：块大小记成掩码本身（192）去对边界。顺带记：本题掩码 255.255.192.0 即 /18，子网内地址范围 183.80.64.0 ~ 183.80.127.255，广播地址是最后一个 183.80.127.255。

【逐项排雷】

A：183.80.0.0——把第三字节整个抹成 0，相当于当成 /16 的掩码，过度清零

B：✓ 72 ∧ 192 = 64，网络地址 183.80.64.0

C：183.80.72.0——你选的：只把最后一字节清零，第三字节没和掩码做与运算，等于只做了"看起来像网络地址"的样子

D：183.80.192.0——把掩码的字节 192 抄成了网络地址，张冠李戴

【一句话结论】 求网络地址就是 IP ∧ 掩码：255 抄、0 写 0、中间字节算与；速算用块大小法，边界取块首。

【同类题指针】 子网划分（求网络地址/广播地址/可用主机数/判断同子网）几乎每年必考一道，常与下一题这种"看图定网关"连环出现，回测时把 35、36 当一组练。

第 36 题 · 看拓扑图定子网掩码与默认网关（你犹豫的那道）

【题目】 按图中网络拓扑，主机 H 的子网掩码与默认网关分别是多少。正确答案 **D（255.255.255.224，192.168.1.62）**。你这次：❌ 错，在 A、C 之间标记——两个选项的网关都写的.1，这正是本题埋的钩子。（图中：H 的 IP 为 192.168.1.60，与它直连的路由器接口是 192.168.1.62/27；.1 是路由器另一侧接口的地址，不在 H 的子网内。）

【考点小块】

小块 1 · 掩码看"和 H 直连的那个路由器接口"。概念：主机所在的这段网，规矩由路由器接口上的标注说了算——接口写 /27，H 的掩码就是 /27 对应的 255.255.255.224（/27 即前 27 位为 1：255.255.255.11100000 = 224）。套路：先在图上找到 H 挂在哪台路由器、哪个接口下，读那个接口的 IP 和前缀长度，掩码直接换算。验证：/27 的块大小 = 256 − 224 = 32，子网边界 0、32、64……；接口.62 和 H 的.60 同落 32~63 这一块（192.168.1.32 ~ 192.168.1.63），同子网 ✓。易错：拿另一个接口的 /26（255.255.255.192）来套——那是另一段网的规矩，A、B 就是这么错的。

小块 2 · 默认网关 = 本子网内路由器接口的地址。概念：网关是"出本子网的门"，必须是**和 H 同一个子网、且在路由器上**的那个接口地址——H 要能直接把帧递到它手里。套路：候选地址逐个验"是否与 H 同子网"：.62 与.60 同属.32/27 子网 ✓；.1 落在 0~31 的另一块里，根本不在 H 的子网，H 连 ARP 都广播不到它，当不了网关。易错（正是你的错）：觉得".1 看着像网关"就选.1——网关不看顺眼不顺眼，只看在不在同一子网。

【逐项排雷】

A：255.255.255.192 +.1——掩码抄了别的接口的 /26，网关又选了外网的.1，双错（你犹豫的两个之一）

B：255.255.255.192 +.62——网关对了，但掩码仍是 /26，别段网的规矩

C：255.255.255.224 +.1——掩码对了，网关.1 不在 H 的子网内，递不到（你犹豫的另一个）

D：✓ 掩码 /27 = 255.255.255.224，网关 = 同子网的路由器接口 192.168.1.62

【一句话结论】 掩码认直连接口的前缀，网关认同子网的路由器接口；.1 再顺眼，不在同子网就不是网关。

【同类题指针】 "看图求掩码+网关"与第 35 题是同一考点（子网划分）的两种问法，回测时连着练；判断两个地址是否同子网，统一用块大小法对边界。

第 37 题 · SDN 的南向接口

【题目】 在 SDN 网络体系结构中，控制器向数据平面的交换机下发流表时，使用的是什么接口？正确答案 **B（南向接口）**。你这次：⬜ 空白——SDN 是个纯概念点，一次记死，以后都是送分。

【考点小块】

小块 1 · SDN 的三层结构先立起来。概念：传统网络里每台交换机自己决定怎么转发；SDN 把"大脑"抽出来集中成**控制器**，交换机只剩"手脚"（数据平面，照流表转发），大脑和手脚之间靠接口对话。整个结构分三层：上面是应用平面（各种网络 App），中间是控制平面（控制器），下面是数据平面（交换机）。套路：凡问 SDN 接口，先定位"谁和谁在说话"，再对方向。

小块 2 · 四个方向一次记死。想象控制器站在中间，拿地图方位记：**南向接口** = 控制器 ↔ 下面的交换机（下发流表就走它，最典型的协议是 OpenFlow）；**北向接口** = 控制器 ↔ 上面的应用程序（给 App 调用，本质是 API）；**东向/西向接口** = 控制器 ↔ 其他控制器（多个控制器之间互通信息，东西不分家，一起记）。易错：把"北向"误当成"向上级汇报所以是控制命令下发"——方向是按被连接的对象在图上的位置定的：命令往下走（到交换机）就是南向，不管语气多强势。

【逐项排雷】

A：东向接口是控制器与控制器之间的，不连交换机

B：✓ 控制器向数据平面交换机下发流表，正是南向接口的定义场景（OpenFlow）

C：西向接口同东向，也是控制器之间互联，不连交换机

D：北向接口朝上，给应用层 App 调用控制器用，方向正好反了

【一句话结论】 SDN 接口看方位：下发流表往下走是南向，给 App 调用往上是北向，控制器之间是东西向。

【同类题指针】 SDN 在 408 里属低频概念点，考法就是这种"某动作走哪个接口"的单点判断，回测时留意即可，不必深挖协议细节。

第 38 题 · TCP 超时后拥塞窗口重新涨到 16KB 的时间

【题目】 甲、乙已建 TCP 连接，MSS = 1 KB，甲一直有数据要发；当拥塞窗口 cwnd = 16 KB 时发生了超时，问 cwnd 再次增长到 16 KB 至少需要多长时间。正确答案 **C（11 RTT）**。你这次：⬜ 空白——这类题是纯推演，有固定剧本，跟着走一遍就会。

【考点小块】

小块 1 · 超时那一刻发生两件事（剧本第一幕）。概念：拥塞控制是 TCP 在"试探路有多宽"：窗口越大发得越猛，超时说明路上堵出事了，立刻收紧。套路（超时处理固定动作）：① 慢启动门限 ssthresh 设为当前 cwnd 的一半：16 ÷ 2 = **8 KB**；② cwnd 直接重置为 1（个 MSS）= **1 KB**，从头再来。易错：只记"减半"忘了 cwnd 要回到 1，或者把 ssthresh 设成 16 本身。

小块 2 · 两个阶段的涨法不同（剧本第二幕）。**慢启动阶段**（cwnd < ssthresh）：每过 1 个 RTT，cwnd 翻一倍（1→2→4→8…），是指数涨；**拥塞避免阶段**（cwnd ⩾ ssthresh）：每过 1 个 RTT，cwnd 只 +1（8→9→10…），是线性涨。套路：先用翻倍冲到门限，再换成 +1 慢慢爬。逐 RTT 推演本题：

起点：cwnd = 1（超时刚重置），ssthresh = 8

第 1 个 RTT 后：1 → 2（慢启动翻倍）

第 2 个 RTT 后：2 → 4

第 3 个 RTT 后：4 → 8（到达门限，慢启动结束，共用 **3 个 RTT**）

之后拥塞避免，每 RTT +1：8 → 9 → 10 → 11 → 12 → 13 → 14 → 15 → 16，共 **8 个 RTT**

合计：3 + 8 = **11 RTT**。易错：冲过门限还继续翻倍（那会算出 4 个 RTT，是 A 的来历）；或全程按 +1 爬（16 个 RTT 上下，是 D 的来历）。

【逐项排雷】

A：4 RTT——当成慢启动一路翻倍到 16（1→2→4→8→16），忘了到门限 8 就得换挡

B：5 RTT——慢启动段数对了一半，但拥塞避免段严重少算，8 涨到 16 靠 +1 需要 8 步，不是 2 步

C：✓ 慢启动 3 RTT（到 8）+ 拥塞避免 8 RTT（8→16），合计 11 RTT

D：16 RTT——当成从 1 开始全程每 RTT +1，没用上慢启动的翻倍

【一句话结论】 超时先两件事：门限减半、窗口归 1；再先翻倍冲到门限、后 +1 爬到目标，两段 RTT 分开数再相加。

【同类题指针】 "超时/收到 3 个冗余 ACK 后窗口如何变、多久涨回"是 TCP 拥塞控制的招牌计算题，各年份反复出现，回测时留意把"3 个冗余 ACK"版本（只减半、不归 1，快重传+快恢复）与本题的超时版本对照记。

第 39 题 · TCP 断开连接：双方进入 CLOSED 的时间

【题目】 客户 C 和服务器 S 已建 TCP 连接，RTT = 50 ms，MSL = 800 ms，数据传完后 C **主动**请求断开。从 C 发出 FIN 的时刻算起，C 和 S 进入 CLOSED 状态所需的时间至少各是多少？正确答案 **D（1650 ms，75 ms）**。你这次：⬜ 空白——又是一道时间线推演题，把每一步的耗时标出来就不难。

【考点小块】

小块 1 · 四次挥手 + 谁主动关谁等 2MSL。概念：断开连接像礼貌告别：一方向另一方说"我没了"（FIN），对方回"知道了"（ACK），对方也说"我也没了"（FIN），第一方再回"知道了"（ACK）。**主动关闭的一方**在发出最后一个 ACK 后不能马上走，要进入 TIME_WAIT 等 **2MSL**——等的原因是怕最后一个 ACK 丢失、对方重发 FIN 时自己已经不在了；被动方只要收到那个 ACK 就能干脆关闭。套路：先认谁主动（本题 C 先发 FIN → C 是主动方，2MSL 的等待记在 C 头上；S 是被动方，不等）。易错：把 2MSL 记到被动方头上，或只记 1 个 MSL（那是 C 选项 850 的来历）。

小块 2 · 沿时间线推演（记单程时延 = RTT/2 = 25 ms）。设 t = 0 为 C 发出 FIN 的时刻，按"最少时间"算（S 没多余数据，收到 FIN 立刻回 ACK 并紧接着发自己的 FIN，可以合并在同一时刻发出）：

t = 0：C 发出 FIN①，C 进入 FIN_WAIT 状态

t = 25 ms（半个 RTT）：S 收到 FIN① → 回 ACK②、同时发 FIN③，进入 LAST_ACK

t = 50 ms（1 个 RTT）：C 收到 ACK② 和 FIN③ → 发出最后的 ACK④，进入 TIME_WAIT 开始等 2MSL

t = 75 ms：S 收到 ACK④ → **S 进入 CLOSED**（总耗时 **75 ms**）

t = 50 + 2×800 = **1650 ms**：C 的 TIME_WAIT 计时结束 → **C 进入 CLOSED**

易错：C 的计时起点不是 t = 0，而是 C 收到对方 FIN、发出最后 ACK 的 t = 50 ms；漏掉这 50 ms 就会算出 1600 ms（无此选项，正是出题人防的）。

【逐项排雷】

A：850 ms，50 ms——C 只等了 1 个 MSL（50+800），S 的时间也少算了最后一段 ACK 的单程

B：1650 ms，50 ms——C 算对了；但 S 不是在 C 发出 ACK 时（t=50）就关闭，而是要等 ACK 传到（t=75）

C：850 ms，75 ms——S 算对了；C 又只等 1 个 MSL，2MSL 记成 1 个

D：✓ C = 50 + 2×800 = 1650 ms；S = 75 ms，两个数都对

【一句话结论】 主动关的一方等 2MSL 且从"发出最后 ACK"起算，被动方收到最后 ACK 即关；所有单程时延按半个 RTT 数。

【同类题指针】 挥手计时与第 38 题同属"TCP 时间线推演"家族，回测时留意 TIME_WAIT 的存在理由（防最后 ACK 丢失）这类配套概念问法。

第 40 题 · HTTP/1.1 取一个网页 + 一张图的总时间

【题目】 主机 H 用 HTTP/1.1 浏览服务器 S 上的页面 news408.html（大小 1 MSS），页面里引用了同目录下 1 幅图像（大小 3 MSS），RTT = 10 ms，忽略首部开销和段传输时延，H 已完成域名解析。从 H 请求与 S 建立 TCP 连接的时刻算起，到收齐全部内容，至少需要多长时间？正确答案 **B（40 ms）**。你这次：⬜ 空白——三道计时题里套路最综合的一道，其实就是"建连 + 两次请求"分段相加。

【考点小块】

小块 1 · HTTP/1.1 的连接规矩。概念：HTTP/1.0 每取一个文件就要新建一次 TCP 连接；**HTTP/1.1 默认持久连接**——建一次连接，多个文件排队在同一条连接上取，不用反复握手。套路：看到 1.1，先确认"握手只花一次"。另外请求只能**串行**：先把 html 取回来、解析出"里面有张图"，才知道要再发第二个请求，所以两次请求不能并行省时间。易错：按 1.0 的思路给图像再算一次建连（多 1 个 RTT，是 C 的来历）；或以为两个请求可以同时发（少算，是 A 的方向）。

小块 2 · 分三段推演（每段按 RTT 数）。注意接收大文件要受 TCP 慢启动约束：连接刚建好时 cwnd = 1 MSS，每过 1 个 RTT 翻倍。

第 1 段 · 建连接：三次握手，从发出 SYN 到能带上请求，计 **1 个 RTT**（第 3 次握手可捎带 HTTP 请求，这是"至少"的算法）

第 2 段 · 取 html：发请求 → 1 MSS 的页面传回，**1 个 RTT**，此时 H 才知道还有张图（累计 2 RTT）

第 3 段 · 取图像（3 MSS）：发请求后，服务器受慢启动限制不能一次全发：先发 1 MSS（cwnd = 1），过 1 个 RTT 后 cwnd 涨到 2，再发 2 MSS，又过 1 个 RTT 收齐——本段 **2 个 RTT**（累计 4 RTT）

合计：1 + 1 + 2 = 4 RTT = 4 × 10 ms = **40 ms**。易错：图像段按"3 MSS 一次发完只花 1 个 RTT"算（忘了慢启动，总时间会算成 30 ms，是 A 的来历）；或把传输时延也算进去（题干已明说忽略）。

【逐项排雷】

A：30 ms——图像按 1 个 RTT 一次传完，漏了慢启动下 3 MSS 要分两批发

B：✓ 建连 1 + html 1 + 图像 2 = 4 个 RTT = 40 ms

C：50 ms——多算了一个 RTT，典型是给图像请求又算了一次建连接（把 1.1 当 1.0）

D：60 ms——在 C 的基础上再多算一段，通常是把域名解析也计了时间，但题干说已完成解析

【一句话结论】 HTTP/1.1 只握手一次、请求串行排队；总时间 = 建连 1 RTT + 每个小文件 1 RTT + 大文件按慢启动分批数 RTT。

【同类题指针】 "HTTP 取页面总耗时"与第 38、39 题同属计网计时推演家族，回测时留意 1.0 与 1.1、流水线与非流水线的变体问法，解法都是先分段、再逐段数 RTT。

回测清单（隔 2 天，10-23 重做）

重做题号：**第 33、34、35、36、37、38、39、40 题（全部 8 题）**。

使用说明：

1. 对的题（33）：不用重做原题，只看确认一遍——想想当时是真懂"相邻结点→链路层"，还是恰好蒙对；排雷里 D 选项的坑（传输层也是流量控制）要能说清为什么不选。

2. 错题（35、36）：先合上这份拆解，自己把原题重做一遍——35 题必须写出 72 ∧ 192 的过程或用块大小法对一遍边界，36 题先写下"网关必须与 H 同子网"再回图里找接口；做完再打开拆解核对。

3. 空白题（34、37、38、39、40）：先只看每题的【考点小块】把套路补上（34 背公式分工、38/39/40 各自背下推演剧本），然后合上材料回做原题，能独立算出答案才算过关；38、39、40 三道计时题建议连着重做，它们共用"分段 + 逐段数 RTT/时延"的同一种脑子。